

Python Basics — Simple Notes

Beginner-friendly explanations with a clear example for every topic.
Clean structure • Consistent format • Ready for practice

Contents

1. Python Fundamentals
2. Conditional Statements
3. Loops
4. Python Data Structures
5. Functions
6. Strings
7. File Handling
8. Exception Handling
9. Modules and Packages
10. Object-Oriented Programming
11. Intermediate Python
12. Python for Cybersecurity
13. Python for AI / ML
14. Recommended Learning Order & Practice

1. Python Fundamentals

Installing Python

Meaning: Putting Python on your computer so you can write and run Python programs.

Example:

```
print("Hello")
```

print()

Meaning: Shows something on the screen.

Example:

```
print("Hello Kuvalesh")
```

Comments

Meaning: Notes for humans. Python does not execute them.

Example:

```
# This is a comment  
print("Hello")
```

Variables

Meaning: A named box that stores information.

Example:

```
name = "Kuvalesh"  
age = 21  
print(name)
```

Naming Rules

Meaning: A variable name cannot start with a number or contain spaces. Use letters, numbers and underscore.

Example:

```
student_name = "Rahul"  
age2 = 20
```

Tip: Use snake_case for variable names (preferred Python style).

int

Meaning: Whole numbers (integers).

Example:

```
age = 21
```

float

Meaning: Numbers with decimals.

Example:

```
price = 99.50
```

str

Meaning: Text inside quotes (single or double).

Example:

```
name = "Kuvalesh"
```

bool

Meaning: Only True or False.

Example:

```
is_student = True
```

None

Meaning: Means there is currently no value.

Example:

```
result = None
```

Type Conversion

Meaning: Changing one data type into another.

Example:

```
age = int("21")
price = float("10.5")
text = str(100)
```

input()

Meaning: Gets information typed by the user (always returns a string).

Example:

```
name = input("Enter your name: ")
print(name)
```

Tip: Convert input with int() or float() when you need numbers.

Arithmetic Operators

Meaning: Do mathematics: + - * // % **

Example:

```
a = 10
b = 3
print(a + b)    # 13
print(a % b)    # 1 (remainder)
print(a // b)   # 3 (floor division)
print(a ** b)   # 1000 (power)
```

Comparison Operators

Meaning: Compare two values and give True or False: == != > < >= <=

Example:

```
print(10 > 5)    # True
print(10 == 10)  # True
print(10 != 5)   # True
```

Logical Operators

Meaning: Combine conditions using and, or, and not.

Example:

```
age = 20
print(age > 18 and age < 30) # True
```

Assignment Operators

Meaning: Store or update values: = += -= *= /= //= %= **=

Example:

```
x = 10
x += 5 # same as x = x + 5
print(x) # 15
```

Membership Operators

Meaning: Check whether something exists inside something else (in / not in).

Example:

```
name = "Kuvalesh"
print("K" in name) # True
```

Identity Operators

Meaning: Check whether two variables refer to the same object (is / is not).

Example:

```
a = [1, 2]
b = a
print(a is b) # True
```

2. Conditional Statements

if

Meaning: Runs code when a condition is true.

Example:

```
age = 20
if age >= 18:
    print("Adult")
```

elif

Meaning: Checks another condition when the previous one was false.

Example:

```
marks = 75
if marks >= 90:
    print("A")
elif marks >= 60:
    print("B")
```

else

Meaning: Runs when all previous conditions are false.

Example:

```
age = 15
if age >= 18:
    print("Adult")
else:
    print("Child")
```

Nested if

Meaning: An if statement inside another if statement.

Example:

```
age = 20
has_id = True
if age >= 18:
    if has_id:
        print("Entry allowed")
```

Ternary Operator

Meaning: A short one-line if-else.

Example:

```
result = "Adult" if age >= 18 else "Child"
```

3. Loops

for loop

Meaning: Repeats code for each item or number in a sequence.

Example:

```
for i in range(5):
    print(i) # 0 1 2 3 4
```

while loop

Meaning: Repeats code while a condition is true.

Example:

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

range()

Meaning: Creates a sequence of numbers, commonly used with loops.

Example:

```
for i in range(1, 6):
    print(i) # 1 2 3 4 5
# range(start, stop, step)
```

break

Meaning: Stops a loop immediately.

Example:

```
for i in range(10):
    if i == 5:
        break
    print(i) # 0 1 2 3 4
```

continue

Meaning: Skips the current loop round and goes to the next.

Example:

```
for i in range(5):
    if i == 2:
        continue
    print(i) # 0 1 3 4
```

pass

Meaning: Does nothing; useful as a placeholder.

Example:

```
if True:
    pass # later you can add code here
```

Nested loops

Meaning: A loop inside another loop.

Example:

```
for i in range(3):
    for j in range(2):
        print(i, j)
```

Practice ideas

Meaning: Common beginner practice: even/odd, factorial, prime numbers, multiplication tables and patterns.

Example:

```
n = 7
print("Even" if n % 2 == 0 else "Odd")
```

4. Python Data Structures

String

Meaning: Stores text (immutable).

Example:

```
name = "Kuvalesh"
```

List

Meaning: Stores multiple values and can be changed (mutable, ordered).

Example:

```
fruits = ["apple", "banana", "mango"]
fruits.append("orange")
```

Tuple

Meaning: Stores multiple values and cannot be changed after creation (immutable, ordered).

Example:

```
numbers = (10, 20, 30)
```

Set

Meaning: Stores unique values; duplicates are automatically removed (unordered).

Example:

```
numbers = {1, 2, 2, 3}
print(numbers) # {1, 2, 3}
```

Dictionary

Meaning: Stores information as key-value pairs (mutable).

Example:

```
student = {"name": "Kuvalesh", "age": 21}
print(student["name"])
```

Indexing

Meaning: Gets an item by its position. Python starts from index 0.

Example:

```
fruits = ["apple", "banana", "mango"]
print(fruits[1]) # banana
```

Slicing

Meaning: Gets part of a sequence: sequence[start:stop:step]

Example:

```
numbers = [10, 20, 30, 40, 50]
print(numbers[1:4]) # [20, 30, 40]
```

Add / Remove / Update

Meaning: Change the contents of a mutable data structure.

Example:

```
fruits.append("orange")
fruits.remove("apple")
fruits[0] = "grapes"
```

Iterating

Meaning: Going through items one by one.

Example:

```
for fruit in fruits:
    print(fruit)
```

5. Functions

Defining and Calling

Meaning: A function is reusable code made to do a particular job.

Example:

```
def greet():
    print("Hello")
greet()
```

Parameters and Arguments

Meaning: A parameter is what a function expects; an argument is the value you give it.

Example:

```
def greet(name):
    print("Hello", name)
greet("Kuvalesh")
```

return

Meaning: Sends a result back from a function.

Example:

```
def add(a, b):
    return a + b
print(add(10, 20)) # 30
```

Default Arguments

Meaning: A parameter already has a value if you do not provide one.

Example:

```
def greet(name="Student"):
    print("Hello", name)
greet() # Hello Student
```

Keyword Arguments

Meaning: Pass a value by writing the parameter name.

Example:

```
def student(name, age):
    print(name, age)
student(age=21, name="Kuvalesh")
```

*args

Meaning: Lets a function receive many positional values as a tuple.

Example:

```
def add(*numbers):
    print(numbers)
add(10, 20, 30) # (10, 20, 30)
```

****kwargs**

Meaning: Lets a function receive many key-value arguments as a dictionary.

Example:

```
def student(**details):  
    print(details)  
student(name="Kuvalesh", age=21)
```

Scope

Meaning: The area where a variable can be used (local vs global).

Example:

```
def test():  
    x = 10 # local  
    print(x)  
test()
```

Lambda

Meaning: A small one-line anonymous function.

Example:

```
square = lambda x: x * x  
print(square(5)) # 25
```

6. Strings

Indexing and Slicing

Meaning: Use positions or ranges to get parts of text.

Example:

```
text = "Python"  
print(text[0]) # P  
print(text[1:4]) # yth
```

split()

Meaning: Breaks a string into a list of pieces.

Example:

```
print("I love Python".split()) # ["I", "love", "Python"]
```

join()

Meaning: Joins multiple strings together.

Example:

```
words = ["I", "love", "Python"]  
print(" ".join(words)) # I love Python
```

replace()

Meaning: Replaces one piece of text with another.

Example:

```
print("I like Java".replace("Java", "Python"))
```

strip()

Meaning: Removes extra spaces (or characters) from the beginning and end.

Example:

```
print(" Kuvalesh ".strip()) # Kuvalesh
```

lower() / upper()

Meaning: Changes text to lowercase or uppercase.

Example:

```
print("HELLO".lower()) # hello  
print("hello".upper()) # HELLO
```

find()

Meaning: Finds the starting position of text (–1 if not found).

Example:

```
print("Python".find("t")) # 2
```

count()

Meaning: Counts how many times a substring appears.

Example:

```
print("banana".count("a")) # 3
```

f-strings

Meaning: Easy way to put variables inside text (Python 3.6+).

Example:

```
name = "Kuvalesh"  
age = 21  
print(f"My name is {name} and I am {age}")
```

7. File Handling

open()

Meaning: Opens a file so Python can work with it. Prefer the with form.

Example:

```
file = open("hello.txt", "r") # older style
```

Reading

Meaning: Gets content from a file.

Example:

```
with open("hello.txt", "r") as file:  
    data = file.read()  
    print(data)
```

Writing

Meaning: Creates or replaces file content (mode 'w').

Example:

```
with open("hello.txt", "w") as file:  
    file.write("Hello Python")
```

Appending

Meaning: Adds content without replacing existing content (mode 'a').

Example:

```
with open("hello.txt", "a") as file:  
    file.write("\nNew line")
```

with open()

Meaning: A safer way to work with files because Python handles closing the file automatically.

Example:

```
with open("hello.txt", "r") as file:  
    print(file.read())
```

TXT, CSV and JSON

Meaning: Python can read and write common text, table and structured-data files.

Example:

```
import json  
data = {"name": "Kuvalesh"}  
text = json.dumps(data)
```

8. Exception Handling

try

Meaning: Contains code that might cause an error.

Example:

```
try:
    x = 10 / 0
```

except

Meaning: Handles the error so the program does not crash.

Example:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

else

Meaning: Runs when no error happens inside the try block.

Example:

```
try:
    x = 10 / 2
except:
    print("Error")
else:
    print("Success")
```

finally

Meaning: Runs whether there is an error or not (cleanup code).

Example:

```
try:
    print("Hello")
finally:
    print("Finished")
```

raise

Meaning: Creates an error intentionally when a rule is broken.

Example:

```
age = 15
if age < 18:
    raise ValueError("Under 18")
```

Common Errors

Meaning: ValueError can happen with a bad value; ZeroDivisionError happens when dividing by zero.

Example:

```
age = int("abc") # ValueError
```

9. Modules and Packages

import

Meaning: Brings a whole module into your program.

Example:

```
import math
print(math.sqrt(25)) # 5.0
```

from ... import

Meaning: Imports a specific item from a module.

Example:

```
from math import sqrt
print(sqrt(25))
```

Creating your own module

Meaning: Put reusable functions in another .py file and import them.

Example:

```
# calculator.py
def add(a, b):
    return a + b

# main.py
from calculator import add
print(add(3, 4))
```

pip

Meaning: Installs Python packages from PyPI.

Example:

```
pip install requests
```

Virtual Environments

Meaning: Keep project dependencies separate from other projects.

Example:

```
python -m venv myenv
# activate: source myenv/bin/activate (Linux/Mac)
#           myenv\Scripts\activate   (Windows)
```

requirements.txt

Meaning: A file listing the packages a project needs.

Example:

```
requests
pandas
numpy
# install with: pip install -r requirements.txt
```

Useful built-in modules

Meaning: math, random, datetime, os, sys, json and re provide common functionality.

Example:

```
import math
import random
import datetime
```

10. Object-Oriented Programming

Class

Meaning: A blueprint used to create objects.

Example:

```
class Student:
    pass
```

Object

Meaning: A thing created from a class (an instance).

Example:

```
student1 = Student()
```

__init__

Meaning: Runs automatically when an object is created (constructor).

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
```

Instance Variables and Methods

Meaning: Variables store object data; methods are functions inside a class.

Example:

```
class Student:
    def __init__(self, name):
        self.name = name
    def greet(self):
        print("Hello", self.name)

s = Student("Kuvalesh")
s.greet()
```

Inheritance

Meaning: A child class can get features from a parent class.

Example:

```
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    pass

d = Dog()
d.eat() # Eating
```

Encapsulation

Meaning: Keeps data and the code that uses it together and can restrict direct access.

Example:

```
class Bank:
    def __init__(self):
        self.__balance = 1000 # name-mangled
```

Polymorphism

Meaning: The same method name can behave differently for different objects.

Example:

```
class Dog:
    def sound(self): print("Bark")
class Cat:
    def sound(self): print("Meow")
```

Abstraction

Meaning: Shows what is needed while hiding complicated internal details.

Example:

```
print("Hello") # You use it without knowing its internal implementation
```

11. Intermediate Python

List Comprehension

Meaning: Short way to create a list.

Example:

```
numbers = [i for i in range(5)] # [0, 1, 2, 3, 4]
```

Dictionary Comprehension

Meaning: Short way to create a dictionary.

Example:

```
squares = {i: i*i for i in range(5)}
```

Set Comprehension

Meaning: Short way to create a set.

Example:

```
squares = {i*i for i in range(5)}
```

Iterators

Meaning: Give values one by one using next().

Example:

```
numbers = iter([10, 20, 30])
print(next(numbers)) # 10
```

Generators

Meaning: Produce values one at a time using yield (memory efficient).

Example:

```
def numbers():
    yield 1
    yield 2

for n in numbers():
    print(n)
```

Decorators

Meaning: Add extra behavior to a function without changing its code.

Example:

```
def decorator(func):
    def wrapper():
        print("Before")
        func()
    return wrapper
```

Regular Expressions

Meaning: Find patterns in text (import re).

Example:

```
import re
result = re.search(r"\d{10}", "9876543210")
```

map()

Meaning: Applies a function to each item of an iterable.

Example:

```
numbers = [1, 2, 3]
print(list(map(lambda x: x*2, numbers))) # [2, 4, 6]
```

filter()

Meaning: Keeps items that satisfy a condition.

Example:

```
numbers = [1, 2, 3, 4]
print(list(filter(lambda x: x % 2 == 0, numbers))) # [2, 4]
```

reduce()

Meaning: Combines items into one result (from functools).

Example:

```
from functools import reduce
print(reduce(lambda a, b: a + b, [1, 2, 3, 4])) # 10
```

zip()

Meaning: Pairs items from multiple sequences.

Example:

```
names = ["A", "B"]
marks = [80, 90]
print(list(zip(names, marks))) # [("A", 80), ("B", 90)]
```

enumerate()

Meaning: Gives both index and value while looping.

Example:

```
for i, name in enumerate(["A", "B"]):
    print(i, name)
```

Context Managers

Meaning: Manage resources safely, commonly with the with statement.

Example:

```
with open("hello.txt") as file:
    data = file.read()
```

Type Hints

Meaning: Show expected data types in code (helps readability and tools).

Example:

```
def add(a: int, b: int) -> int:
    return a + b
```

Dataclasses

Meaning: Make simple data-holding classes easier to write.

Example:

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    age: int
```

12. Python for Cybersecurity

os

Meaning: Interact with the operating system (paths, directories, environment).

Example:

```
import os
print(os.getcwd())
```

sys

Meaning: Access Python runtime and command-line information.

Example:

```
import sys
print(sys.version)
```

subprocess

Meaning: Run system commands from Python.

Example:

```
import subprocess
subprocess.run(["ls"])
```

socket

Meaning: Build network communication between programs.

Example:

```
import socket
s = socket.socket()
```

requests

Meaning: Send HTTP requests to websites and APIs.

Example:

```
import requests
r = requests.get("https://example.com")
print(r.status_code)
```

json

Meaning: Work with structured data commonly used by APIs.

Example:

```
import json
data = {"name": "Kuvalesh"}
```

re

Meaning: Find patterns in text — useful for log and data analysis.

Example:

```
import re
print(re.findall(r"\d+", "ID 123")) # ["123"]
```

hashlib

Meaning: Create hashes for integrity and security-related tasks.

Example:

```
import hashlib
print(hashlib.sha256(b"hello").hexdigest())
```

base64

Meaning: Encode or decode data (it is not encryption).

Example:

```
import base64
print(base64.b64encode(b"hello"))
```

scapy

Meaning: Analyze and create network packets for controlled network experimentation.

Example:

```
from scapy.all import IP
packet = IP()
```

logging

Meaning: Record events, warnings and errors.

Example:

```
import logging
logging.warning("Something happened")
```

File handling for security

Meaning: Read logs and files for analysis and integrity-related tools.

Example:

```
with open("app.log", "r") as file:
    logs = file.read()
```

13. Python for AI / ML

NumPy

Meaning: Works with numbers, arrays and mathematical operations.

Example:

```
import numpy as np
numbers = np.array([1, 2, 3])
print(numbers * 2) # [2 4 6]
```

Pandas

Meaning: Works with tables and datasets (DataFrames).

Example:

```
import pandas as pd
df = pd.DataFrame({"Name": ["A", "B"], "Age": [20, 21]})
print(df)
```

Matplotlib

Meaning: Creates graphs and charts.

Example:

```
import matplotlib.pyplot as plt
plt.plot([1, 2, 3], [10, 20, 30])
plt.show()
```

Scikit-learn

Meaning: Provides tools for machine learning models.

Example:

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

PyTorch / TensorFlow

Meaning: Frameworks used for deep learning and advanced AI.

Example:

```
import torch
# or: import tensorflow as tf
```

14. Recommended Learning Order & Practice

Suggested sequence:

1. Python basics (variables, types, operators)
2. Data structures (list, tuple, set, dict)
3. Functions
4. File handling
5. Exception handling
6. Modules & packages
7. Object-Oriented Programming
8. JSON and Regular Expressions
9. OS and subprocess
10. Networking (socket, requests)
11. Scapy (controlled experiments)
12. NumPy and Pandas
13. Machine Learning basics

Practice Target

Meaning: Solve 50–100 small Python problems before moving heavily into advanced cybersecurity or AI/ML libraries.

Example:

Start with: even/odd, factorial, prime number, calculator, palindrome, counting characters.

Best Practice

Meaning: Write programs yourself instead of only watching tutorials. Build a small project after each major section.

Example:

Learn loops → make a number guessing game

Learn files → make a simple notes app

Learn requests → make a simple API project

Keep coding every day. Small consistent practice beats long irregular sessions.